FUNCTION REFERENCE 0.8

API function description with examples.

See last pages for complete examples for browser and node.

ON SYNCHRONOUS CALLS

The core functions – put() and get() and others – return a promise. There are synchronous variants for most of them, that add 'Sync' to the function name. They can make sense for fast prototyping as they can often be used in synchronous functions. But there are limitations.

Note that synchronous functions do not work with node and not when connecting to a network, as opposed to working with an in-memory POT. The difference is set by using new() with or without its optional parameters (with parameters: network, without: in-memory).

INITIALIZATION

NEW MAP

Create a new Swarm map.

`pot.new([<bee url>, <batch id>]) → promise of map`

In-memory (for testing):

```
(async () => {
      await pot.ready()
      map = await pot.new()
      ...
})()
```

Connecting to a Swarm network. In the example, a local network.

```
(async () => {
```

```
        await pot.ready()

        map = await pot.new("http://localhost:1633",
 "6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa")

        ...
})()
```

pot.newSync() → map

Synchronous creation of a map (mostly for prototyping):

```
(async () => {

        await pot.ready()

        map = pot.newSync()

        ...
})()
```

NEW MAP FROM SAVED REFERENCE

Create a new Swarm KVS from a reference.

pot.newByReference(reference : int) → promise of map

```
map = await pot.new()

await map.put(key1, val1)

ref = await map.save()

map2 = await pot.newByReference(ref)
```

pot.newByReferenceSync(reference : int) → map

Synchronous creation of a map (mostly for prototyping).

The other functions in this example might as well be async.

```
map = pot.newSync()

map.putSync(key1, val1)
```

```
ref = map.saveSync()

map2 = pot.newByReferenceSync(ref)
```

SAVE

map.save() → promise of reference

map.saveSync() → reference

Store a Swarm map to the network. Not meaningful for in-memory settings.

Put actions always operate in-memory first, until the entire map is saved to the network.

Examples see, above, **newByReference()**

STORING AND RETRIEVING

The following two are the preferred functions for real-world use.

PUT TYPED

Put a value with a code for its type in the 1st byte of the raw value.

get() automatically converts it back to the right type. getRaw() returns all bytes, including the first, undecoded, as Uint8Array.

map.put(key : any, value : any) → promise of null

```
map = await pot.new()

try {

    await map.put(“fox”, "The quick brown fox jumps!")

} catch(e) {
    ...
}
```

map.putSync(key : any, value : any) → error or null

Store synchronously, with leading type encoding byte. Mostly for prototyping.

GET TYPED

Get a value from the POT, interpreting its first byte as type information.

map.get (key : any) ⟶ promise of boolean, number or string

```
try {

        value = await map.get(key)

} catch(e) {
        ...
}
```

When using **.catch**, don't chain it before taking the reference; assign it first.

```
ref = pot.hangingPromise()
ref.catch((err)=>attestExpectedError(T, err, "Error: canceled"))
```

map.getSync(key : any) ⟶ boolean, number or string

Synchronously get the value decoded for its type using the 1st byte of the raw value.

Mostly for prototyping.

ON LOADED

pot.ready() ⟶ Promise of null

Delay until the pot object is initialized.

```
await pot.ready()

map = await pot.new()

await map.put("hello", "POT!")
```

onWasmLoaded() ⟶ any

This function is called from Go, if it exists, when the initialization of the pot object is done.

```
function onWasmLoaded() {

        map = pot.newSync()

        map.putSync("hello", "POT!")

        value = map.getSync("hello")

        console.log("key ‹hello› has:", value)
```

```
}
```

RAW BYTE ACCESS

To store Uint8Arrays directly, or to inspect the pure bytelevel beneaththe type-encoding, values can be stored as 'raw' bytes without the leading, type encoding byte.

These values can be retrieved raw, by getRaw(), or by having them converted immediately to a desired type with getBoolean(), getNumber(), and getString(). Retrieving values stored with putRaw() with get() or getSync() is most of the time not sensical as they would interprete the fist byte as type information and then discard it.

STORE RAW

`map.putRaw(key : any, value : Uint8Array) → promise null or Error`

`map.putRawSync(key : any, value : Uint8Array) → null or Error`

Put a raw `Uint8Bytes` array.

RETRIEVE FROM RAW

`map.getRaw(key : any) → Promise of Uint8Array`

`map.getRawSync(key : any) → Uint8Array`

Get a raw `Uint8Bytes` array.

`map.getBoolean(key : any) → Promise of boolean`

`map.getBooleanSync(key : any) → boolean`

Get the raw value as Boolean. First byte 0 being false, all else true.

DO NOT USE ON BOOLEAN VALUE STORED WITH PUT() OR PUTSYNC().

`map.getNumber(key : any) → Promise of number`

`map.getNumberSync(key : any) → number`

Get the raw value as floating point number (JS default).

DO NOT USE ON NUMBER VALUE STORED WITH PUT() OR PUTSYNC().

`map.getString(key : any) → Promise of string`

`map.getStringSync(key : any) → string`

Get the raw value as string.

DO NOT USE ON STRING VALUE STORED WITH PUT() OR PUTSYNC().

CRYPTO

NOT IMPLEMENTED.

GET PROOF

pot.getProof(hash32) hash32

Get a proof for an entry.

TESTING AND SUPPORT

These functions exist for black box testing. See `kvs_test.js` for examples of their use.

LOG

pot.log(message : string)

Write to browser console via Go to verify language-lands roundtrip.

HELLO

pot.hello()

Write hello to browser console to verify established WASM stream.

RANDOM KEY

pot.randKey() → Uint8Array(32)

32 random bytes for key testing.

RANDOM VALUE

pot.randValue() → Uint8Array(79–101)

79 to 101 random bytes for value testing (range taken from Go POT).

TYPE ENCODED BYTES

pot.type_encoded_bytes(value : any) → Uint8Array

JS type detection and value encoding with correct leading type byte.

TYPE DECODED VALUE

pot.type_decoded_value(raw : Uint8Array) → boolean, number, or string

JS type detection of raw kvs value by leading type byte.

HANGING PROMISE

pot.hangingPromise() → promise of null

Blocking promise (in Go) with 1s time out for exception testing.

SET FAULT

pot.setFaul()

Make the next function call to the mock storage function return an (JS) error to JS.

SET PANIC

pot.setPanic()

Make the next function call to the mock storage function panic (on the Go side).

SET DELAY

pot.setDelay(msec : int)

Make the next function call to the mock storage function stall for msec milliseconds.

SET HANG

pot.setHang(msec : int)

Make the next function call to the mock storage function stall indefinitely.

EXAMPLES

The following examples, except for the last, are for in-browser programming. The last one illustrates how POT JS runs in node.

EXAMPLE 1

A simple put and get, logging to console.

example1.html

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>

(async () => {

	await pot.ready()

	map = await pot.new()

	await map.put("hello", "POT!")

	value = await map.get("hello")

	console.log("key ‹hello› has:", value)
})()

</script>
```

EXAMPLE 2: INTERACTION

Interactive use.

example2.html

```
<pre>

	POT JS Example 2: Interactive & Integrity

	key    <input id=key>

	value  <input id=value>
```

```
              <button onclick="put()"> put </button>


<hr />


       key    <input id=search>

              <button onclick="get()"> get </button>

       value  <input id=result>


       <img src=favicon.ico>


</pre>

<script src="wasm_exec.js" integrity="sha384-PWCs+V4BDf9yY1yjkD/p+9xNEs4iE-buvq+HezAOJiY3XL5GI6VyJXMsvnjiwNbce"></script>
<script src="potjs.js" integrity="sha384-dUfhLIycF6GUprYxIlI-iAAQR3rh3KsPxPdsgq4tP2rTe7grND7bXjAj4nawjq5Ht"></script>

<script>

       var map

       (async () => {

              await pot.ready()
              map = await pot.new()

       })()

       const field = (id) => { return document.getElementById(id) }

       async function put() {

              key = field('key').value
              value = field('value').value
              await map.put(key, value)
       }

       async function get() {

              key = field('search').value
              value = await map.get(key)
              field('result').value = value
       }

</script>
```

EXAMPLE 3: INITIALIZATION

More explicit initialization.

This page does not use pot.js but initializes pot.wasm directly.

example3.html

```
<pre>

	POT JS Example 3: Explicit Initialization

	key	<input id=key>
	value	<input id=value>
		<button onclick="put()"> put </button>

<hr />

	key	<input id=search>
		<button onclick="get()"> get </button>
	value	<input id=result>

</pre>
<script src="wasm_exec.js"></script>
<script>
	const go = new Go()
	var map;
	WebAssembly.instantiateStreaming(fetch("pot.wasm"), go.importObject)
		.then((r)=>{
			go.run(r.instance)
			map = pot.newSync()
		})
	const field = (id) => { return document.getElementById(id) }
	async function put() {
		key = field('key').value
		value = field('value').value
		await map.putSync(key, value)
	}
	async function get() {
		key = field('search').value
		value = await map.getSync(key)
		field('result').value = value
	}
</script>
```

EXAMPLE 4: SYNCHRONOUS

This page uses synchronous calls that can be helpful for prototyping but don't work for network connections (only in-memory = new() without parameters) and not for use with node.

example4.html

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>

function onWasmLoaded() {

	map = pot.newSync()

	map.putSync("hello", "POT!")

	value = map.getSync("hello")

	console.log("key ‹hello› has:", value)
}

</script>
```

EXAMPLE 5: NETWORK

This page connects to a local Swarm network.

The example is for illustration, the batch id would have to be updated. See example5.html (and 6).

example5.html

```
<script src="wasm_exec.js"></script>
<script src="potjs.js"></script>

<script>
(async () => {

	await pot.ready()

	map = await pot.new("http://localhost:1633",
	"6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa")

	await map.put("hello", "POT!")

	value = await map.get("hello")

	console.log("key ‹hello› has:", value)

	ref = await map.save()

	console.log("tree root is:", ref)

})()

</script>
```

EXAMPLE 6: NETWORK II

Example 6 works like example 5 but can be run directly with `make example6`.

See **example6.html**

EXAMPLE 7: NODE

example7.js

.

```
fs = require("fs");
require("./wasm_exec"); // note the ./

var go = new Go();

WebAssembly.instantiate(fs.readFileSync("pot.wasm"), go.importObject)
	.then((r) => { go.run(r.instance) })

onWasmLoaded = async () => {

	console.log("wasm loaded-function called")

	map = await global.pot.new()

	await map.put("hello", "node")

	value = await map.get("hello")

	console.log("hello:", value)
}
```